

Python Master Notes

PART 6 - Practical Python, DSA & Complexity

From Python knowledge to real programming: modules, files, the standard library, data structures, problem-solving patterns, complexity analysis, competitive-programming technique, and 50 fully solved practice problems.

Section	What you get
1. Modules & Packages	imports, aliases, packages, <code>__name__</code> , <code>__main__</code>
2. File Handling	modes, <code>with</code> , context managers, text vs binary, JSON
3. Standard Library	math, random, datetime, collections, itertools, functools, heapq, bisect, os, sys
4. Python for DSA	list, string, stack, queue, deque, hash map/set, linked list, heap, recursion, sorting
5. DSA Patterns	14 patterns with signals, templates and examples
6. Complexity	Big-O/Theta/Omega, cases, amortized, Python operation costs
7. Interview & CP	fast IO, built-ins, mistakes, choosing a data structure
8. Template Sheet	copy-ready code templates
9. Complexity Cheat Sheet	one-page tables
10. 50 Practice Problems	beginner -> intermediate -> advanced, all solved and tested

KEY IDEA: How to use this part

Read a section, then close the notes and rewrite its template from memory. Every solution in Section 10 was executed against test cases before being printed here, so you can copy-run them safely. Try each problem yourself for 15-20 minutes before reading the solution.

TIP: Prerequisite

Parts 1-5: references and mutability, iterators and generators, exceptions, and OOP. This part assumes you are comfortable with lists, dicts, sets, functions and comprehensions.

1. Modules & Packages

1.1 Module, package, library - the vocabulary

Term	Meaning	Example
Module	A single .py file containing definitions and code	math, your own utils.py
Package	A folder of modules (usually with __init__.py)	collections, json
Library	A collection of packages/modules you install or that ship with Python	standard library, NumPy
Import	Statement that loads a module and binds a name to it	import math

1.2 Import forms

```
import math                # bind the module; use math.sqrt(9)
from math import sqrt, pi  # bind specific names; use sqrt(9)
import math as m           # alias; use m.sqrt(9)
from math import sqrt as rt # alias for one name
from collections import Counter, defaultdict, deque
from math import *         # AVOID: floods your namespace, hides where names come from
```

Style	Good for	Downside
import x	clarity - you always see x.func	a bit longer to type
from x import y	short code in DSA/CP (heappush(...))	name clashes possible
import x as y	long names (import numpy as np)	keep to well-known aliases
from x import *	quick REPL experiments only	pollutes namespace; unclear origin

1.3 What actually happens on import

- Python checks the cache `sys.modules`. If the module is already there, it is reused (no re-run).
- Otherwise it searches the folders in `sys.path` (current folder, standard library, site-packages).
- It executes the module file top to bottom **once**, creating the module object.
- It binds the name in your current namespace.

```
import sys
print(sys.path)          # list of folders Python searches
print("math" in sys.modules) # True after you import math
import math
print(math.__name__)      # 'math'
print(dir(math)[:5])      # names inside the module
help(math.gcd)            # built-in documentation
```

COMMON TRAP: Shadowing the standard library

If you name your file `random.py`, `math.py` or `json.py`, then `import random` may load YOUR file instead of the real one, causing confusing errors. Never name files after standard modules.

COMMON TRAP: Circular imports

If `a.py` imports `b.py` and `b.py` imports `a.py`, one of them sees a half-initialised module. Fix by moving shared code to a third module, or importing inside the function that needs it.

1.4 Packages

```
myproject/
  main.py
  utils.py
  shapes/
    __init__.py    # a package
                  # runs when the package is imported (can be empty)
    circle.py
    rectangle.py
```

```
# in main.py
import utils
from shapes import circle
from shapes.rectangle import area
import shapes.circle as sc
```

- `__init__.py` marks a folder as a regular package and can expose a clean public API.
- Absolute import: `from shapes.circle import area`. Relative import (inside a package): `from .circle import area`.
- Install third-party packages with `pip install name`; use a virtual environment (`python -m venv .venv`) per project.

1.5 `__name__` and `if __name__ == "__main__"`

Every module has a built-in variable `__name__`. When the file is run directly, Python sets it to `"__main__"`. When the file is imported, it is set to the module's name.

```
# tools.py
def add(a, b):
    return a + b

print("tools loaded, __name__ =", __name__)

if __name__ == "__main__":
    print(add(2, 3))          # runs ONLY with: python tools.py
```

How the file is used	<code>__name__</code> is	Guarded block runs?
<code>python tools.py</code>	<code>"__main__"</code>	Yes
<code>import tools</code> from another file	<code>"tools"</code>	No

TIP: Standard program layout

Put logic in functions, then `def main(): ...` and end the file with the `__main__` guard. This makes your code importable and testable, and is exactly what interviewers expect in real projects.

2. File Handling

2.1 Opening files and file modes

```
f = open("notes.txt", "r", encoding="utf-8")
data = f.read()
f.close()                                # you MUST close - so prefer `with` (section 2.3)
```

Mode	Meaning	If file missing	Existing content
r	read (default)	error	kept
w	write	created	ERASED
a	append (write at end)	created	kept
x	exclusive create	created	error if it exists
r+	read and write	error	kept, pointer at start
w+	write and read	created	ERASED
b	binary flag: rb, wb, ab	-	bytes instead of str
t	text flag (default): rt	-	str

COMMON TRAP: w destroys data instantly

Opening an existing file with "w" truncates it to zero length the moment it is opened - even if you never write anything. Use "a" to add, or "x" to be safe.

2.2 Reading and writing

```
with open("notes.txt", "r") as f:
    whole = f.read()           # entire file as ONE string
with open("notes.txt") as f:
    first = f.readline()       # one line (includes the trailing \n)
with open("notes.txt") as f:
    lines = f.readlines()      # list of all lines (loads everything)
with open("notes.txt") as f:
    for line in f:             # BEST for big files: lazy, one line at a time
        print(line.rstrip("\n"))

with open("out.txt", "w") as f:
    f.write("hello\n")         # write does NOT add a newline for you
    f.writelines(["a\n", "b\n"])
    print("via print", file=f) # print adds the newline

with open("out.txt", "a") as f:
    f.write("appended line\n")
```

Method	Returns	Note
read()	str (whole file)	read(n) reads n characters
readline()	str (one line)	empty string "" at end of file
readlines()	list of str	keeps \n at the end of each item
for line in f	iterates lines	memory efficient
write(s)	number of chars	only accepts str (or bytes in wb)
seek(0) / tell()	move / report position	rewind to re-read

2.3 The with statement and context managers

with guarantees cleanup (closing the file) even if an exception happens inside the block. Any object that implements `__enter__` and `__exit__` is a **context manager**.

```
# roughly what `with open(...) as f:` does
f = open("a.txt")
try:
    data = f.read()
```

```

finally:
    f.close()

# Your own context manager - class version
class Timer:
    def __enter__(self):
        import time
        self.t0 = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        import time
        print("elapsed:", time.perf_counter() - self.t0)
        return False
        # False = do not swallow exceptions

with Timer():
    sum(range(10**6))

# generator version using contextlib
from contextlib import contextmanager

@contextmanager
def opened(path):
    f = open(path)
    try:
        yield f
    finally:
        f.close()
        # code inside `with` runs here

```

You can open several files in one statement:

```

with open("in.txt") as src, open("out.txt", "w") as dst:
    for line in src:
        dst.write(line.upper())

```

2.4 Text vs binary files

	Text mode (r, w)	Binary mode (rb, wb)
Data type	str	bytes
Decoding	automatic using encoding	none - raw bytes
Newlines	translated (\r\n <-> \n)	untouched
Use for	.txt .csv .json .py	images, PDFs, zip, audio, pickles

```

# copy any file (works for images, PDFs...)
with open("photo.png", "rb") as src, open("copy.png", "wb") as dst:
    dst.write(src.read())

text = "cafe"
b = text.encode("utf-8")      # str -> bytes
print(b, b.decode("utf-8"))  # bytes -> str

```

TIP: Always pass encoding="utf-8" in text mode

The default encoding depends on the operating system. Being explicit avoids 'works on my laptop, fails on the server' bugs.

2.5 JSON basics

JSON is the universal text format for data exchange (APIs, config files). Python's `json` module converts between JSON text and Python objects.

```

import json

data = {"name": "Pikach", "year": 2, "skills": ["C++", "Python"], "cgpa": None}

s = json.dumps(data, indent=2)      # Python -> JSON string
back = json.loads(s)                # JSON string -> Python

with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)     # Python -> file
with open("data.json", encoding="utf-8") as f:
    obj = json.load(f)               # file -> Python

```

Python	JSON
dict	object {}
list, tuple	array [] (a tuple comes back as a list!)
str	string
int, float	number
True / False	true / false
None	null

COMMON TRAP: JSON gotchas

Dictionary keys always become strings ({1: 'a'} -> {"1": "a"}). Sets and custom objects are not serialisable by default. dumps/loads work with strings; dump/load work with files (remember: the trailing s = string).

2.6 Handling file errors

```
try:
    with open("missing.txt") as f:
        data = f.read()
except FileNotFoundError:
    print("file not found")
except PermissionError:
    print("no permission")
except json.JSONDecodeError as e:      # when parsing bad JSON
    print("bad json:", e)
```

3. The Standard Library You Must Know

Python ships with 'batteries included'. For DSA and interviews the most valuable modules are collections, heapq, bisect, itertools, functools and math.

3.1 math

Function / constant	Purpose	Example -> result
<code>sqrt(x)</code>	square root (float)	<code>sqrt(16)</code> -> 4.0
<code>isqrt(n)</code>	exact integer square root	<code>isqrt(17)</code> -> 4
<code>ceil(x), floor(x)</code>	round up / down (return int)	<code>ceil(2.1)</code> -> 3
<code>gcd(a, b), lcm(a, b)</code>	greatest common divisor / least common multiple	<code>gcd(12, 18)</code> -> 6
<code>factorial(n), comb(n, k), perm(n, k)</code>	combinatorics (exact ints)	<code>comb(5, 2)</code> -> 10
<code>pow(a, b, m)</code> (built-in)	modular exponentiation, fast	<code>pow(2, 10, 1000)</code> -> 24
<code>log(x), log2(x), log10(x)</code>	logarithms	<code>log2(8)</code> -> 3.0
<code>prod(iterable)</code>	product of elements	<code>prod([2, 3, 4])</code> -> 24
<code>inf, pi, e</code>	constants	<code>float('inf')</code> is equivalent to <code>math.inf</code>
<code>isclose(a, b)</code>	compare floats safely	<code>isclose(0.1 + 0.2, 0.3)</code> -> True

COMMON TRAP: Never compare floats with ==

`0.1 + 0.2 == 0.3` is False. Use `math.isclose`. Also / always returns a float; use `//` for integer division. For big numbers avoid `int(a / b)` - float precision is lost above about 10^{15} .

3.2 random

```
import random
random.seed(42)                # reproducible results (great for testing)
random.random()                # float in [0, 1)
random.randint(1, 6)           # int, BOTH ends inclusive
random.randrange(0, 10, 2)     # like range(): stop excluded
random.choice([10, 20, 30])    # one random element
random.choices([1, 2, 3], k=5) # with replacement
random.sample(range(100), 5)   # without replacement
arr = [1, 2, 3, 4]
random.shuffle(arr)             # in place, returns None
```

Use random to generate test data and stress-test your solution against a brute force. It is NOT secure - use the secrets module for passwords/tokens.

3.3 datetime and time

```
from datetime import date, datetime, timedelta

today = date.today()
now = datetime.now()
d = date(2026, 9, 28)
print(d.year, d.month, d.day, d.weekday()) # weekday(): Monday = 0
print(d + timedelta(days=30))              # date arithmetic
delta = date(2026, 12, 31) - d              # timedelta
print(delta.days)

print(now.strftime("%Y-%m-%d %H:%M"))      # datetime -> string
dt = datetime.strptime("28/09/2026", "%d/%m/%Y") # string -> datetime

import time
t0 = time.perf_counter()                  # best timer for benchmarking
# ... code ...
print(time.perf_counter() - t0)
```

Code	Meaning	Code	Meaning
%Y	4-digit year	%H	hour (24h)
%m	month 01-12	%M	minute
%d	day 01-31	%S	second
%B	month name	%A	weekday name

3.4 collections (the big three + two)

```
from collections import Counter, defaultdict, deque, namedtuple, OrderedDict

c = Counter("mississippi")          # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
c.most_common(2)                   # [('i', 4), ('s', 4)]
c["z"]                             # 0 (missing key gives 0, no KeyError)
Counter("aab") + Counter("abc")     # add counts
Counter("aab") - Counter("abc")     # subtract (drops zero/negative)

g = defaultdict(list)              # missing key -> [] automatically
g["a"].append(1)
n = defaultdict(int)               # missing key -> 0
n["x"] += 1

dq = deque([1, 2, 3])
dq.appendleft(0)                   # O(1) at BOTH ends
dq.popleft()
dq.rotate(1)                        # rotate right by 1

Point = namedtuple("Point", "x y") # lightweight immutable record
pt = Point(3, 4)
print(pt.x, pt[1])
```

- Counter - frequency counting, anagrams, top-K, multiset comparison.
- defaultdict - graph adjacency lists, grouping, avoiding if key not in d.
- deque - queues, BFS, sliding-window maximum, O(1) both ends.
- OrderedDict - rarely needed now (plain dict keeps insertion order); useful for move_to_end (LRU caches).

3.5 itertools

```
from itertools import (permutations, combinations, product, accumulate, chain,
                      groupby, islice, count, cycle, zip_longest,
                      combinations_with_replacement)

list(permutations([1, 2, 3], 2))    # ordered pairs: 6 of them
list(combinations([1, 2, 3], 2))   # [(1,2), (1,3), (2,3)]
list(product([0, 1], repeat=3))    # all 8 binary strings of length 3
list(accumulate([1, 2, 3, 4]))     # [1, 3, 6, 10] = prefix sums
list(chain([1, 2], [3], (4,)))     # flatten several iterables
list(islice(count(1), 5))          # [1, 2, 3, 4, 5] (count is infinite)
for key, grp in groupby(sorted("aabbcc")): # groups CONSECUTIVE equal items
    print(key, len(list(grp)))      # so SORT first
list(zip_longest([1, 2, 3], "ab", fillvalue="-"))
```

CP / INTERVIEW: itertools shortcuts

accumulate = prefix sums in one line. product(range(n), repeat=k) replaces k nested loops. combinations produces C(n,k) items - only use when n is tiny (n <= 20 or so).

3.6 functools

```
from functools import lru_cache, cache, reduce, partial, cmp_to_key

@lru_cache(maxsize=None)           # memoization: cache results by arguments
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
fib(80)                            # instant; without the cache it would take ages
# @cache is the short form of lru_cache(maxsize=None) (Python 3.9+)

reduce(lambda a, b: a * b, [1, 2, 3, 4]) # 24: fold a list into one value
```

```
double = partial(pow, 2)                                # fix the first argument: double(5) = 32

# custom comparison sort: largest number by concatenation
nums = [3, 30, 34, 5, 9]
nums.sort(key=cmp_to_key(lambda a, b: -1 if str(a) + str(b) > str(b) + str(a) else 1))
print("".join(map(str, nums)))                          # 9534330
```

TIP: Memoization = top-down DP

Arguments must be hashable (ints, strings, tuples - not lists). `fib.cache_clear()` resets it. Deep recursion can still hit the recursion limit (Section 4.10).

3.7 heapq

```
import heapq
h = []
heapq.heappush(h, 5)
heapq.heappush(h, 1)
heapq.heappush(h, 3)
h[0]                                # smallest element, O(1) peek -> 1
heapq.heappop(h)                    # removes and returns 1, O(log n)

a = [5, 1, 3, 9, 2]
heapq.heapify(a)                    # in-place, O(n)
heapq.nlargest(2, a)                # [9, 5]
heapq.nsmallest(2, a)               # [1, 2]
heapq.heappush(h, (2, "task"))      # tuples compare by first item, then next...
```

heapq is a **min-heap** only. For a max-heap store negative values (`heappush(h, -x)`).

3.8 bisect

```
import bisect
arr = [1, 3, 3, 5, 7]               # MUST be sorted
bisect.bisect_left(arr, 3)          # 1 first index where arr[i] >= 3
bisect.bisect_right(arr, 3)         # 3 first index where arr[i] > 3
bisect.insort(arr, 4)                # insert keeping order (O(n) because of shifting)
count_3 = bisect.bisect_right(arr, 3) - bisect.bisect_left(arr, 3) # occurrences of 3
```

3.9 os, pathlib and sys

```
import os, sys
os.getcwd()                        # current folder
os.listdir(".")                    # names in a folder
os.path.join("data", "in.txt")     # portable path building
os.path.exists("in.txt"), os.path.isfile("in.txt"), os.path.getsize("in.txt")
os.makedirs("out/logs", exist_ok=True)
os.environ.get("HOME")              # environment variables
os.remove("old.txt")                # delete a file

from pathlib import Path            # modern, object-style paths
p = Path("data") / "in.txt"
p.exists(); p.read_text(encoding="utf-8"); p.suffix; p.parent

sys.argv                            # command-line arguments (list of str)
sys.exit(0)                         # quit the program
sys.setrecursionlimit(10**6)        # raise recursion limit (default about 1000)
sys.stdin.readline()                # fast input
sys.getsizeof([1, 2, 3])           # bytes used by an object
sys.maxsize                         # largest 'index-size' int
```

Module	Reach for it when...	Star functions
math	numeric work, gcd, combinatorics	gcd isqrt comb inf
random	test data, simulation	randint shuffle sample
datetime	dates and time differences	date timedelta strftime
collections	counting, queues, grouping	Counter defaultdict deque
itertools	combinatorics, prefix sums, chaining	accumulate product permutations
functools	memoization, custom sort	cache reduce cmp_to_key

Module	Reach for it when...	Star functions
heapq	priority queue, top-K, Dijkstra	heappush heappop nlargest
bisect	binary search on sorted lists	bisect_left bisect_right
os / pathlib	files, folders, environment	os.path.join Path
sys	fast IO, recursion limit, argv	stdin setrecursionlimit exit

4. Python for DSA

A data structure is a way of organising data so that the operations you need are fast. This section shows the practical Python implementation of each one.

4.1 Arrays / lists

A Python list is a **dynamic array**: contiguous references to objects that grows automatically. Index access is $O(1)$; inserting or deleting in the middle is $O(n)$ because elements shift.

```
arr = [5, 2, 9, 1]
arr[0], arr[-1]          # first, last
arr[1:3]                  # slice copy [2, 9]
arr.append(7)              #  $O(1)$  amortized
arr.insert(0, 4)           #  $O(n)$ 
arr.pop()                 #  $O(1)$  last; arr.pop(0) is  $O(n)$ !

n = 5
zeros = [0] * n           # 1D array of size n
grid = [[0] * m for _ in range(n)] # 2D n x m (NEVER [[0]*m]*n)
for i in range(n):
    for j in range(m):
        grid[i][j] = i * j

transpose = [list(r) for r in zip(*grid)]
flat = [x for row in grid for x in row]
```

COMMON TRAP: `[[0]*m]*n`

This makes n references to the SAME inner list. Changing one row changes all rows. Always use the comprehension `[[0]*m for _ in range(n)]`.

Traversal idioms

```
for x in arr: ...          # values only
for i in range(len(arr)): ... # indices
for i, x in enumerate(arr): ... # both
for i in range(len(arr) - 1, -1, -1): ... # backwards
for a, b in zip(arr, arr[1:]): ... # adjacent pairs
```

4.2 Strings

Strings are **immutable**. Every 'modification' builds a new string. To build a string piece by piece, collect parts in a list and join once ($O(n)$), instead of `s += ch` in a loop (can degrade to $O(n^2)$).

```
s = "hello world"
s[::-1]                # reverse
s.split()              # ['hello', 'world']
"".join(reversed(s))   # reverse via join
chars = list(s)        # make it mutable
chars[0] = "H"
s2 = "".join(chars)

ord("a"), chr(97)       # 97, 'a'
ord("d") - ord("a")    # 3 -> index of a letter in 0..25
count = [0] * 26
for ch in "banana":
    count[ord(ch) - ord("a")] += 1 # fixed-size frequency array

s.isalpha(), s.isdigit(), s.isalnum(), s.islower()
s.strip(), s.lstrip("h"), s.rstrip()
s.find("wor"), s.index("wor")    # find returns -1 if absent; index raises ValueError
"abc" < "abd"                   # strings compare lexicographically
```

4.3 Stack (LIFO)

Last In, First Out. A Python list is a perfect stack: `append` pushes, `pop` pops, `[-1]` peeks - all $O(1)$.

```
stack = []
stack.append(1)          # push
stack.append(2)
top = stack[-1]          # peek (check `if stack:` first)
```

```

x = stack.pop()          # pop -> 2
if not stack: print("empty")

class Stack:
    def __init__(self):
        self.data = []
    def push(self, x):
        self.data.append(x)
    def pop(self):
        return self.data.pop()          # raises IndexError if empty
    def peek(self):
        return self.data[-1]
    def __len__(self):
        return len(self.data)

```

Use a stack for: matching brackets, undo/redo, evaluating expressions, DFS, monotonic-stack problems, reversing things, and turning recursion into iteration.

4.4 Queue (FIFO) and 4.5 deque

First In, First Out. Do **not** use `list.pop(0)` as a queue: it is $O(n)$. Use `collections.deque`, which supports $O(1)$ operations at both ends.

```

from collections import deque
q = deque()
q.append(1)          # enqueue at right
q.append(2)
front = q[0]         # peek front
x = q.popleft()      # dequeue from left -> 1, O(1)
q.appendleft(0)      # push at left (deque = double-ended queue)
q.pop()              # pop from right
len(q), bool(q)

d = deque([1, 2, 3, 4, 5])
d.rotate(2)          # [4, 5, 1, 2, 3]
w = deque(maxlen=3)  # fixed-size window: old items fall off automatically
for x in range(6):
    w.append(x)
print(list(w))       # [3, 4, 5]

```

WATCH OUT: deque indexing

`q[0]` and `q[-1]` are $O(1)$ but `q[i]` in the middle is $O(n)$. If you need random access, use a list. `queue.Queue` exists for multi-threading (it has locking overhead) - do not use it for DSA.

4.6 Hash map (dict)

A dict stores key -> value pairs in a **hash table**: a hash function turns the key into a bucket index, so lookup, insert and delete are $O(1)$ on average. Collisions (two keys, one bucket) are resolved internally. Keys must be **hashable** (immutable): int, str, tuple of hashables. Since Python 3.7 dicts keep insertion order.

```

d = {}
d["a"] = 1
d["b"] = d.get("b", 0) + 1          # safe counting
"a" in d                            # O(1) key membership
d.pop("a", None)
for k, v in d.items(): ...
d.setdefault("k", []).append(5)    # create list if missing, then append

from collections import Counter, defaultdict
freq = Counter([1, 1, 2])
graph = defaultdict(list)
graph[1].append(2)
graph[2].append(1)                 # undirected edge 1-2

seen = {}                          # value -> index (Two Sum style)
pos = {ch: i for i, ch in enumerate("abc")}

```

4.7 Hash set

```

s = set()
s.add(3)

```

```

s.discard(10)                # no error if absent (remove() raises KeyError)
3 in s                        # O(1) average
a, b = {1, 2, 3}, {2, 3, 4}
a & b, a | b, a - b, a ^ b    # intersection, union, difference, symmetric diff
a <= b                        # subset test
visited = set()              # classic use: visited nodes / seen values
visited.add((r, c))          # tuples are hashable -> grid coordinates
fs = frozenset([1, 2])      # immutable set (hashable) - can be a dict key

```

KEY IDEA: Empty set / dict

`{}` is an empty **dict**. An empty set must be `set()`. A set holds unique, hashable items and has no order.

4.8 Linked list concepts

A linked list is a chain of **nodes**; each node stores a value and a reference to the next node. Python rarely needs one in real code (lists and deques are better), but interviews test it heavily.

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# build 1 -> 2 -> 3
head = ListNode(1, ListNode(2, ListNode(3)))

# traverse
cur = head
while cur:
    print(cur.val)
    cur = cur.next

# insert at head - O(1)
head = ListNode(0, head)

# dummy-head technique: removes special cases for the first node
def remove_val(head, target):
    dummy = ListNode(0, head)
    cur = dummy
    while cur.next:
        if cur.next.val == target:
            cur.next = cur.next.next    # unlink
        else:
            cur = cur.next
    return dummy.next

# find middle: slow/fast pointers
def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

```

Operation	Array-list	Singly linked list
Access by index	O(1)	O(n)
Insert/delete at head	O(n) for <code>insert(0, x)</code>	O(1)
Insert/delete after a known node	O(n)	O(1)
Search	O(n)	O(n)
Memory	compact	extra pointer per node

Core techniques: dummy head, slow/fast pointers (middle, cycle detection), reversing with `prev/cur/nxt` (Problem 35), merging two sorted lists. Always save `cur.next` **before** you overwrite it.

4.9 Heap / priority queue

A heap gives you the smallest (or largest) element in O(1) and lets you insert/remove in O(log n). It is a complete binary tree stored inside a list: children of index *i* are at *2i+1* and *2i+2*.

```

import heapq
h = []
for x in [7, 2, 9, 4]:
    heapq.heappush(h, x)
print(heapq.heappop(h))          # 2 (smallest first)

# max-heap by negating
mh = []
for x in [7, 2, 9, 4]:
    heapq.heappush(mh, -x)
print(-heapq.heappop(mh))       # 9

# priority queue of (priority, item)
pq = []
heapq.heappush(pq, (2, "write code"))
heapq.heappush(pq, (1, "wake up"))
print(heapq.heappop(pq))        # (1, 'wake up')

# heap sort
def heap_sort(a):
    heapq.heapify(a)
    return [heapq.heappop(a) for _ in range(len(a))]

```

COMMON TRAP: Tuples with un-comparable items

If two priorities tie, Python compares the next tuple element. If that is a custom object it raises `TypeError`. Add a unique tie-breaker: `(priority, counter, item)`.

4.10 Recursion

A function that calls itself on a smaller problem. Every recursive function needs (1) a **base case** that stops it and (2) a **recursive case** that moves toward the base case.

```

def factorial(n):
    if n <= 1:                # base case
        return 1
    return n * factorial(n - 1) # recursive case

def power(a, n):
    # fast exponentiation, O(log n)
    if n == 0:
        return 1
    half = power(a, n // 2)
    return half * half if n % 2 == 0 else half * half * a

def fib_slow(n):
    # O(2^n): recomputes the same subproblems
    return n if n < 2 else fib_slow(n - 1) + fib_slow(n - 2)

from functools import lru_cache
@lru_cache(None)
def fib(n):
    # O(n): each subproblem computed once
    return n if n < 2 else fib(n - 1) + fib(n - 2)

```

- **Call stack:** each call adds a frame; depth n means $O(n)$ space.
- **Recursion limit:** default about 1000 frames -> `RecursionError`. Raise with `sys.setrecursionlimit(10**6)` or convert to an explicit stack/loop.
- **Overlapping subproblems** (Fibonacci) -> memoize. **Independent subproblems** (merge sort) -> plain recursion.
- **Tail recursion is not optimised** in Python; write loops for long chains.

TIP: Recursion checklist

1) What is the smallest input (base case)? 2) How do I shrink the problem? 3) Trust that the recursive call solves the smaller problem. 4) Combine the result. 5) Can it overflow the stack?

4.11 Sorting

Python's `sort()` / `sorted()` use **Timsort**: $O(n \log n)$ worst case, $O(n)$ on nearly-sorted data, and **stable** (equal elements keep their original order). Use the built-in in interviews unless asked to implement a sort.

```
a = [5, 2, 9, 1]
```

```

a.sort() # in place, returns None
b = sorted(a, reverse=True) # new list
people = [("Asha", 21), ("Ravi", 19), ("Meena", 21)]
people.sort(key=lambda t: (-t[1], t[0])) # age descending, then name ascending
words = ["bb", "a", "ccc"]
words.sort(key=len)
sorted("hello") # list of chars; "".join(sorted("hello")) -> 'ehllo'

```

Know how to write the classic algorithms:

```

def insertion_sort(a): # O(n^2) worst, O(n) best; good for tiny/nearly sorted
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a

def merge_sort(a): # O(n log n) always, O(n) extra space, stable
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    L, R = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            out.append(L[i])
            i += 1
        else:
            out.append(R[j])
            j += 1
    return out + L[i:] + R[j:]

def quick_sort(a): # O(n log n) average, O(n^2) worst, not stable
    if len(a) <= 1:
        return a
    pivot = a[len(a) // 2]
    left = [x for x in a if x < pivot]
    mid = [x for x in a if x == pivot]
    right = [x for x in a if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

def counting_sort(a, max_val): # O(n + k) for small integer ranges
    cnt = [0] * (max_val + 1)
    for x in a:
        cnt[x] += 1
    out = []
    for v, c in enumerate(cnt):
        out.extend([v] * c)
    return out

```

Algorithm	Best	Average	Worst	Space	Stable
Bubble / Insertion / Selection	$O(n) / O(n) / O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	B, I: yes; S: no
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	no
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	no
Counting sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	yes
Timsort (Python)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes

5. Common DSA Patterns

Most problems are variations on a small set of patterns. The skill is **recognising the signal** in the problem statement, then filling in the template.

If the problem says...	Try this pattern
count / frequency / duplicates / anagram	Frequency map (Counter, set)
pair with target sum in unsorted array	Hash map (Two Sum)
sorted array, pair/triple, palindrome, in-place partition	Two pointers
longest/shortest contiguous subarray/substring with a condition	Sliding window
sum of a range, many range queries, subarray sum = k	Prefix sum (+ hash map)
sorted data, 'minimum X such that...', search space	Binary search
overlapping, meetings, closest pairs, 'arrange to...'	Sort, then scan
next greater/smaller element, nested brackets, histogram	Stack / monotonic stack
shortest path in unweighted graph/grid, level by level	BFS (queue)
explore all paths, connected components, tree problems	DFS
generate all subsets/permutations/combinations, constraints	Backtracking
k largest/smallest, running median, merge k lists, scheduling	Heap
ranges [start, end]	Intervals (sort by start)
locally best choice is provably globally best	Greedy

5.1 Frequency map

Idea: count occurrences once, then answer questions in $O(1)$. **Signal:** anagram, duplicates, most common, first unique, 'at most k distinct'.

```
from collections import Counter
def freq_template(arr):
    freq = Counter(arr)
    for x in arr:
        if freq[x] == 1:
            return x
    return None
# or manual: freq[x] = freq.get(x, 0) + 1
# first non-repeating
# anagram check: Counter(a) == Counter(b) (Problem 3)
# 26-letter array is faster: cnt[ord(c) - 97] += 1
```

Complexity: $O(n)$ time, $O(k)$ space where k = number of distinct values.

5.2 Two Sum (hash-map lookup)

Idea: for each number x , the partner you need is $\text{target} - x$. Look it up in a dict of numbers already seen. This turns an $O(n^2)$ double loop into $O(n)$.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []
# value -> index
# add AFTER checking (so x cannot pair with itself)
# nums=[2,7,11,15], target=9 -> i=0: need 7 not seen, store 2. i=1: need 2 seen -> [0,1]
```

Variants: count pairs (use a Counter), pairs with difference k , 3-Sum (sort + two pointers), 'subarray sum = k ' (prefix sums + hash map).

5.3 Two pointers

Idea: two indices move through the data, so each element is visited a constant number of times. **Signals:** sorted input, pair sums, palindromes, removing duplicates, merging, partitioning.

```

# (a) opposite ends - sorted array pair sum
def pair_sum_sorted(a, target):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target:
            return [l, r]
        if s < target:
            l += 1                # need bigger sum
        else:
            r -= 1                # need smaller sum
    return []

# (b) same direction (reader/writer) - in-place filtering
def remove_value(a, val):
    w = 0
    for r in range(len(a)):
        if a[r] != val:
            a[w] = a[r]
            w += 1
    return w                # new length

# (c) fast/slow - linked-list cycle, middle

```

Complexity: $O(n)$ time (after $O(n \log n)$ if you must sort first), $O(1)$ space.

5.3b Sliding window

Idea: maintain a window [left, right] over contiguous data; expand right, shrink left when the window becomes invalid. **Signal:** 'longest/shortest/number of **contiguous** subarray or substring such that...'.

```

# FIXED size k: maximum sum of any k consecutive numbers
def max_sum_k(a, k):
    window = sum(a[:k])
    best = window
    for i in range(k, len(a)):
        window += a[i] - a[i - k]    # add new element, drop the old one
        best = max(best, window)
    return best

# VARIABLE size: longest substring with at most k distinct characters
from collections import defaultdict
def longest_k_distinct(s, k):
    cnt = defaultdict(int)
    left = best = 0
    for right, ch in enumerate(s):
        cnt[ch] += 1
        while len(cnt) > k:           # window invalid -> shrink
            cnt[s[left]] -= 1
            if cnt[s[left]] == 0:
                del cnt[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best

```

KEY IDEA: Why it is $O(n)$

left and right each only move forward, so together they make at most $2n$ moves - the inner while does not make it $O(n^2)$.

COMMON TRAP: Sliding window fails with negatives

For 'subarray sum = k' or 'sum $\geq$ k' with negative numbers the window is not monotonic. Use prefix sums + hash map instead.

5.4 Prefix sum

Idea: $\text{prefix}[i] = \text{sum of the first } i \text{ elements}$. Then the sum of $a[l..r] = \text{prefix}[r+1] - \text{prefix}[l]$ in $O(1)$.

Signals: many range-sum queries; 'number of subarrays with sum k'; equal-sum splits.

```

from itertools import accumulate
a = [2, 4, 6, 8, 10]
prefix = [0] + list(accumulate(a))    # [0, 2, 6, 12, 20, 30]
range_sum = lambda l, r: prefix[r + 1] - prefix[l]
range_sum(1, 3)                        # 4+6+8 = 18

```

```
# count subarrays whose sum equals k (works with negatives)
def count_sum_k(a, k):
    seen = {0: 1} # prefix value -> how many times seen
    cur = ans = 0
    for x in a:
        cur += x
        ans += seen.get(cur - k, 0) # earlier prefix = cur - k => subarray sums to k
        seen[cur] = seen.get(cur, 0) + 1
    return ans

# 2D prefix sum: P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] - P[i][j]
```

Also: **difference arrays** apply range updates in $O(1)$ each (add at l , subtract at $r+1$, then prefix-sum once).

5.5 Binary search

Idea: on sorted data (or any monotonic yes/no condition) discard half the search space each step: $O(\log n)$.

Signals: sorted array, 'find the smallest x such that...', 'minimise the maximum', rotated arrays.

```
# classic
def bsearch(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

# lower bound: first index with a[i] >= target (same as bisect_left)
def lower_bound(a, target):
    lo, hi = 0, len(a) # hi = len(a) means 'not found / insert at end'
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo

# binary search on the ANSWER: smallest speed k so that all piles are eaten in h hours
def min_speed(piles, h):
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        hours = sum((p + mid - 1) // mid for p in piles) # ceil(p / mid)
        if hours <= h: # feasible -> try smaller
            hi = mid
        else:
            lo = mid + 1
    return lo
```

CP / INTERVIEW: Binary search on the answer

If 'feasible(x)' is monotonic (false...false true...true), binary-search x and write a feasible check function. Works for capacity, speed, distance, time problems. Use $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) // 2$ in C++/Java to avoid overflow; Python ints never overflow.

COMMON TRAP: Off-by-one

Pick ONE convention and stick to it: $\text{lo} \leq \text{hi}$ with $\text{hi} = \text{mid} - 1$, or $\text{lo} < \text{hi}$ with $\text{hi} = \text{mid}$. Mixing them causes infinite loops. Test with arrays of size 0, 1 and 2.

5.6 Sorting + scanning

Idea: sorting puts related items next to each other, so one linear scan finds what a nested loop would. **Signals:** duplicates, closest pair, overlapping intervals, 'minimum arrows/rooms', anagram groups.

```
def min_diff(a):
    a = sorted(a) # smallest difference between any two numbers
    return min(b - x for x, b in zip(a, a[1:]))
```

```
def largest_number(nums):
    # arrange numbers to form the biggest value
    from functools import cmp_to_key
    s = sorted(map(str, nums), key=cmp_to_key(lambda x, y: -1 if x + y > y + x else 1))
    return "".join(s)

# sort by multiple keys, then scan
students = [("A", 90), ("B", 85), ("C", 90)]
students.sort(key=lambda t: (-t[1], t[0]))
```

Cost: $O(n \log n)$ for the sort dominates the $O(n)$ scan.

5.7 Stack and monotonic stack

Idea (stack): the most recent unmatched item is the one you need first. **Idea (monotonic stack):** keep the stack in increasing or decreasing order; popping tells you the 'next greater/smaller element' for the popped item in amortized $O(1)$.

```
# next greater element for each position (indices on the stack, values decreasing)
def next_greater(a):
    res = [-1] * len(a)
    st = []
    for i, x in enumerate(a):
        while st and a[st[-1]] < x:      # x is the next greater for everything smaller on top
            res[st.pop()] = x
        st.append(i)
    return res
# next_greater([2, 1, 2, 4, 3]) -> [4, 2, 4, -1, -1]

# balanced brackets
def balanced(s):
    pair = {"(": ")", "[": "]", "{": "}"
    st = []
    for c in s:
        if c in pair:
            if not st or st.pop() != pair[c]:
                return False
        else:
            st.append(c)
    return not st
```

Signals for monotonic stack: 'next greater', 'previous smaller', 'daily temperatures', 'largest rectangle in histogram', 'stock span'. **Complexity:** $O(n)$ - every element is pushed and popped at most once.

5.8 Queue and BFS

Idea: explore level by level using a queue. The first time BFS reaches a node is via a **shortest path (fewest edges)**. **Signals:** shortest path in an unweighted graph or grid, minimum moves/steps, level-order, multi-source spread (rotting oranges).

```
from collections import deque

def bfs(graph, start):
    # graph: dict node -> list of neighbours
    # dict doubles as the visited set
    dist = {start: 0}
    q = deque([start])
    while q:
        u = q.popleft()
        for v in graph[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

# grid BFS - 4 directions
def grid_bfs(grid, sr, sc):
    R, C = len(grid), len(grid[0])
    seen = {(sr, sc)}
    q = deque([(sr, sc, 0)])
    while q:
        r, c, d = q.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] != "#" and (nr, nc) not in seen:
```

```

        seen.add((nr, nc))
        q.append((nr, nc, d + 1))
    return seen

```

Complexity: $O(V + E)$ for graphs; $O(R \times C)$ for grids. Multi-source BFS: push all sources first.

COMMON TRAP: Mark visited when you enqueue

If you only mark a node visited when you dequeue it, the same node can be pushed many times (slower, sometimes wrong distances).

5.9 DFS

Idea: go as deep as possible, then backtrack. **Signals:** connected components, flood fill, cycle detection, path existence, tree problems (depth, LCA, path sums), topological order.

```

# graph DFS - recursive
def dfs(graph, u, seen):
    seen.add(u)
    for v in graph[u]:
        if v not in seen:
            dfs(graph, v, seen)

# graph DFS - iterative (avoids RecursionError on deep graphs)
def dfs_iter(graph, start):
    seen = {start}
    st = [start]
    while st:
        u = st.pop()
        for v in graph[u]:
            if v not in seen:
                seen.add(v)
                st.append(v)
    return seen

# binary tree traversals
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def inorder(root):
    # left, node, right (sorted order for a BST)
    return inorder(root.left) + [root.val] + inorder(root.right) if root else []

def height(root):
    return 0 if root is None else 1 + max(height(root.left), height(root.right))

def level_order(root):
    # BFS on a tree
    if not root:
        return []
    out, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        out.append(level)
    return out

```

Complexity: $O(V + E)$. Space $O(V)$ for the visited set plus recursion depth.

5.10 Recursion and backtracking

Idea: build a solution step by step; at each step **choose**, **explore**, then **un-choose** (undo) so the next branch starts clean. Prune branches that cannot succeed. **Signals:** 'all subsets / permutations / combinations', N-Queens, Sudoku, word search, partitioning strings.

```

def backtrack_template(nums):
    res, path = [], []
    def bt(start):

```

```

        res.append(path[:])                # record a COPY (path keeps changing)
        for i in range(start, len(nums)):
            path.append(nums[i])            # choose
            bt(i + 1)                       # explore
            path.pop()                     # un-choose
    bt(0)
    return res                             # all subsets

def combos_sum_k(nums, k):                 # combinations of size k
    from itertools import combinations
    return list(combinations(nums, k))     # or write the backtracking by hand

def generate_parentheses(n):
    res = []
    def bt(s, opened, closed):
        if len(s) == 2 * n:
            res.append(s)
            return
        if opened < n:
            bt(s + "(", opened + 1, closed)
        if closed < opened: # prune: never close more than opened
            bt(s + ")", opened, closed + 1)
    bt("", 0, 0)
    return res

```

COMMON TRAP: Appending path instead of path[:]

res.append(path) stores a reference; after backtracking finishes, every entry is an empty list. Copy it.

Complexity: typically exponential: subsets $O(2^n \times n)$, permutations $O(n! \times n)$. Fine for n up to ~20 and ~10 respectively.

5.11 Heap and top-K

Idea: keep only the K best items in a min-heap of size K ; anything worse than the heap's top is discarded. $O(n \log k)$ beats sorting $O(n \log n)$ when k is small. **Signals:** 'k-th largest', 'top k frequent', 'k closest points', 'merge k sorted', 'running median', scheduling by priority.

```

import heapq
from collections import Counter

def kth_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)            # drop the smallest -> keeps the k largest
    return h[0]

def top_k_frequent(nums, k):
    f = Counter(nums)
    return heapq.nlargest(k, f, key=f.get)

def k_closest(points, k):             # points closest to origin
    return heapq.nsmallest(k, points, key=lambda p: p[0]**2 + p[1]**2)

```

5.12 Intervals

Idea: sort by start time, then sweep while tracking the current end. **Signals:** merge overlapping, insert interval, meeting rooms, non-overlapping selection.

```

def merge(intervals):
    intervals.sort()
    out = [intervals[0]]
    for s, e in intervals[1:]:
        if s <= out[-1][1]:             # overlaps the last merged interval
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out

def max_non_overlapping(intervals):    # activity selection: sort by END time
    intervals.sort(key=lambda x: x[1])

```

```

count, end = 0, float("-inf")
for s, e in intervals:
    if s >= end:
        count += 1
        end = e
return count

```

```

def can_attend_all(intervals):
    intervals.sort()
    return all(a[1] <= b[0] for a, b in zip(intervals, intervals[1:]))

```

Overlap test for $[a, b]$ and $[c, d]$: $a < d$ and $c < b$ (touching endpoints count as overlap only if the problem says so).

5.13 Greedy basics

Idea: make the locally best choice at every step and never reconsider. It only works when a **greedy-choice property** holds - so justify it (exchange argument) or test against brute force. **Signals:** 'minimum number of', 'maximum number of', scheduling, coins with canonical denominations, jump game.

```

def can_jump(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True

def max_profit_many(prices):
    return sum(max(0, b - a) for a, b in zip(prices, prices[1:]))

def min_platforms(arr, dep):
    arr.sort()
    dep.sort()
    i = j = cur = best = 0
    while i < len(arr):
        if arr[i] <= dep[j]:
            cur += 1
            i += 1
            best = max(best, cur)
        else:
            cur -= 1
            j += 1
    return best

```

WATCH OUT: Greedy is not always correct

Coins [1, 3, 4] and amount 6: greedy picks 4+1+1 (3 coins) but the optimum is 3+3 (2 coins). When greedy fails, use dynamic programming (Problem 43).

6. Complexity Analysis

6.1 Why it matters

Complexity describes how the running time (or memory) of an algorithm **grows** as input size n grows. It ignores constant factors and hardware speed - it tells you how the algorithm scales, which is what matters for large inputs and what interviewers actually test.

6.2 Big-O, Big-Omega, Big-Theta

Notation	Meaning	Answers
$O(f(n))$ - Big-O	upper bound: the algorithm never does WORSE than this	'at most how slow?'
$\Omega(f(n))$ - Big-Omega	lower bound: the algorithm never does BETTER than this	'at least how fast?'
$\Theta(f(n))$ - Big-Theta	tight bound: both O and Omega meet	'exactly this rate'

In interviews people say 'Big-O' loosely to mean the **tight** bound (Theta) of the typical/worst case. Formally: $f(n)$ is $O(g(n))$ if there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$ - i.e. $g(n)$ is an upper ceiling on $f(n)$ once n is large enough.

```
# f(n) = 3n^2 + 5n + 2 is O(n^2):  
# drop constants (3) and lower-order terms (5n, 2) - only the fastest-growing term survives
```

6.3 Best, average, and worst case

Case	Meaning	Example: linear search for x in arr
Best case	the friendliest possible input	x is the first element -> $O(1)$
Average case	typical/random input	x is somewhere in the middle -> $O(n)$
Worst case	the input that hurts most	x is the last element or missing -> $O(n)$

Interviews almost always want the **worst case** unless you are asked otherwise. Quicksort is a classic example where average ($O(n \log n)$) and worst ($O(n^2)$) differ sharply.

6.4 Amortized complexity

Some operations are usually cheap but occasionally expensive; **amortized** analysis averages the cost over a long sequence of operations, rather than looking at any single call in isolation.

KEY IDEA: Dynamic array append is $O(1)$ amortized

`list.append` is usually $O(1)$. When the underlying array is full, Python allocates a new array (commonly about 1.125x bigger) and copies everything - one $O(n)$ operation. But that expensive resize happens rarely enough that spread across n appends, the average cost per append is still $O(1)$.

Same idea applies to hash table resizing, and to the two-pointer / sliding-window patterns (each pointer moves forward at most n times in total, so a loop that looks nested is really $O(n)$).

6.5 Space complexity

How much **extra** memory an algorithm uses relative to input size (usually excluding the input itself). Includes extra arrays/hash maps AND the recursion call stack.

```
def sum_iter(arr):  
    # O(1) extra space  
    total = 0  
    for x in arr:  
        total += x  
    return total  
  
def sum_rec(arr):  
    # O(n) extra space - n stack frames  
    if not arr:  
        return 0  
    return arr[0] + sum_rec(arr[1:]) # also O(n) time per slice -> O(n^2) time overall!
```

COMMON TRAP: Slicing costs time AND space

`arr[1:]` copies the list - $O(n)$ time and space per call. The recursive `sum_rec` above is actually $O(n^2)$ time because of this, not $O(n)$. Pass an index instead of slicing when recursing over arrays.

6.6 Complexity of common Python operations

Structure	Operation	Average	Worst case
list	index <code>a[i]</code> , <code>len(a)</code>	$O(1)$	$O(1)$
list	append / pop (end)	$O(1)$ amortized	$O(n)$ on resize
list	insert / pop / delete (front or middle)	$O(n)$	$O(n)$
list	<code>x in a</code> (membership)	$O(n)$	$O(n)$
list	slicing <code>a[i:j]</code>	$O(k)$ (k = slice size)	$O(k)$
list	<code>sort()</code> / <code>sorted()</code>	$O(n \log n)$	$O(n \log n)$
dict	get / set / delete / in	$O(1)$	$O(n)$ (rare, hash collisions)
set	add / remove / in	$O(1)$	$O(n)$ (rare)
deque	append / appendleft / pop / popleft	$O(1)$	$O(1)$
deque	<code>d[i]</code> middle access	$O(n)$	$O(n)$
string	concatenation <code>a + b</code>	$O(\text{len}(a) + \text{len}(b))$	same
string	in, find, count	$O(n)$	$O(n)$
heapq	push / pop	$O(\log n)$	$O(\log n)$
heapq	peek <code>h[0]</code> , heapify	$O(1)$ / $O(n)$	same
bisect	<code>bisect_left/right</code>	$O(\log n)$	$O(\log n)$
<code>sum()</code> <code>max()</code> <code>min()</code> on n items	-	$O(n)$	$O(n)$

COMMON TRAP: `x in list` vs `x in set`

Both LOOK the same in code but a list membership test is $O(n)$ while a set/dict membership test is $O(1)$ on average. Converting a list to a set once ($O(n)$) before doing many lookups is a huge, common speed-up.

6.7 Rule-of-thumb growth chart

n	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
10	~3	10	~33	100	1,024
1,000	~10	1,000	~10,000	1,000,000	astronomical
10^6	~20	10^6	~ 2×10^7	10^{12} (too slow)	impossible

CP / INTERVIEW: Picking an algorithm from constraints

$n \leq 10-12$: any exponential/backtracking is fine. $n \leq 20$: bitmask DP (2^n). $n \leq 1,000$: $O(n^2)$ is fine. $n \leq 10^5$ or 10^6 : need $O(n \log n)$ or $O(n)$. $n \leq 10^9$: need $O(\log n)$ or $O(\sqrt{n})$ or $O(1)$ math. As a rough guide, most judges run about 10^8 simple operations per second.

7. Interview & Competitive Programming Technique

7.1 Fast input handling (competitive programming)

`input()` is convenient but slow for thousands of lines. For large inputs, read everything at once.

```
import sys
input = sys.stdin.readline          # shadow input() with a faster version

n = int(input())
arr = list(map(int, input().split()))

data = sys.stdin.read().split()    # read the WHOLE input at once - fastest
it = iter(data)
n = int(next(it))
arr = [int(next(it)) for _ in range(n)]

print(*arr, sep=" ")               # fast-ish output
sys.stdout.write(" ".join(map(str, arr)) + "\n") # fastest for huge output
```

Task	Slow-ish way	Fast way
read n ints on one line	<code>[int(x) for x in input().split()]</code>	<code>list(map(int, input().split()))</code>
read many lines	repeated <code>input()</code>	<code>sys.stdin.read().split()</code> once
print many values	print in a loop	build a list, one <code>print/join</code> at the end

7.2 Useful built-ins interviewers like seeing

Built-in	What it saves you from writing
<code>enumerate(arr)</code>	manual index counters
<code>zip(a, b)</code>	manual index-pairing of two lists
<code>sorted(x, key=..., reverse=...)</code>	custom sort loops
<code>any(...)/all(...)</code>	manual boolean loops with early return
<code>Counter, defaultdict</code>	manual dictionary bookkeeping
<code>itertools.accumulate</code>	manual prefix-sum loop
<code>math.gcd, math.comb</code>	manual number-theory helper functions
<code>bisect_left/right</code>	manual binary search
f-strings <code>f"{x:.2f}"</code>	<code>%</code> -formatting or <code>.format()</code>

7.3 Common mistakes (and how they show up as bugs)

Mistake	Symptom	Fix
<code>[[0]*m]*n</code>	editing one row edits all rows	<code>[[0]*m for _ in range(n)]</code>
mutable default argument <code>def f(x=[])</code>	list grows across calls	default to <code>None</code> , create inside
<code>==</code> on floats	unexpected <code>False</code>	<code>math.isclose(a, b)</code>
off-by-one in range/slicing	wrong count / <code>IndexError</code>	trace through <code>n = 0, 1, 2</code> by hand
modifying a list while iterating it	skipped elements	iterate over a copy, or build a new list
<code>list.pop(0)</code> in a loop	silently very slow ($O(n)$ each)	use <code>collections.deque</code>
forgetting <code>return</code> in a recursive branch	function returns <code>None</code>	check every branch returns
string concatenation <code>s += c</code> in a loop	$O(n^2)$ time	append to a list, <code>' '.join(...)</code> once
shallow copy of nested structures	inner lists still shared	<code>copy.deepcopy</code> or rebuild manually

Mistake	Symptom	Fix
<code>except:</code> (bare)	hides real bugs, catches <code>KeyboardInterrupt</code> too	catch the specific exception

COMMON TRAP: Mutable default arguments

`def add(x, lst=[]):` the SAME list object is reused across every call that doesn't pass `lst`. Always write `def add(x, lst=None): lst = [] if lst is None else lst`.

7.4 How to choose a data structure quickly

You need to...	Use
look something up by key, fast	dict
test membership / remove duplicates	set
keep order and allow index access	list
add/remove from BOTH ends fast	deque
always grab the min or max quickly	heap (heapq)
undo the most recent action / match nesting	stack (list)
process in the order received / shortest path (unweighted)	queue (deque) + BFS
keep data sorted and query ranges	sort once + bisect, or a heap
represent connections between items	dict of lists (adjacency list)
group items by a key	<code>defaultdict(list)</code>
record an immutable coordinate/state	tuple (hashable -> can go in a set/dict)

7.5 A practical problem-solving checklist

- **Restate** the problem in your own words; confirm input ranges, edge cases (empty, one element, duplicates, negatives).
- **Ask about constraints** (n up to 10? 10^5 ? 10^9 ?) - this tells you the required complexity (Section 6.7).
- **Start with brute force**, state its complexity, THEN optimise - don't jump straight to a clever trick.
- **Name the pattern** (Section 5) out loud: 'this looks like sliding window because...'.
- **Code the template**, trace it on the given example by hand.
- **State final complexity** (time and space) without being asked.
- **Test edge cases**: empty input, single element, all-same elements, already sorted/reverse sorted, negative numbers.

TIP: Talk while you code

Interviewers grade your reasoning as much as your syntax. Narrate assumptions, trade-offs and why you picked a structure - silence is the single most common reason a correct-looking solution scores low.

8. DSA Template Sheet

Copy-ready skeletons. Keep this page open while practicing until each template comes out of memory.

```
# ---- Two pointers (opposite ends) ----
l, r = 0, len(a) - 1
while l < r:
    if condition(a[l], a[r]):
        ...
    elif need_bigger:
        l += 1
    else:
        r -= 1

# ---- Sliding window (variable size) ----
left = 0
state = {}
for right, x in enumerate(a):
    add(x, state)
    while invalid(state):
        remove(a[left], state)
        left += 1
    best = max(best, right - left + 1)

# ---- Prefix sum ----
prefix = [0]
for x in a:
    prefix.append(prefix[-1] + x)
range_sum = lambda l, r: prefix[r + 1] - prefix[l]

# ---- Binary search (exact value) ----
lo, hi = 0, len(a) - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == target: return mid
    elif a[mid] < target: lo = mid + 1
    else: hi = mid - 1

# ---- Binary search on the answer ----
lo, hi = LOW, HIGH
while lo < hi:
    mid = (lo + hi) // 2
    if feasible(mid): hi = mid
    else: lo = mid + 1
return lo

# ---- BFS (graph / grid) ----
from collections import deque
q = deque([start]); seen = {start}
while q:
    u = q.popleft()
    for v in neighbours(u):
        if v not in seen:
            seen.add(v); q.append(v)

# ---- DFS (recursive) ----
def dfs(u, seen):
    seen.add(u)
    for v in neighbours(u):
        if v not in seen:
            dfs(v, seen)

# ---- Backtracking ----
def bt(path, choices):
    if is_solution(path):
        res.append(path[:]); return
    for c in choices:
        path.append(c)           # choose
        bt(path, next_choices)  # explore
        path.pop()              # un-choose

# ---- Monotonic stack (next greater element) ----
stack = []
```

```

for i, x in enumerate(a):
    while stack and a[stack[-1]] < x:
        res[stack.pop()] = x
    stack.append(i)

# ---- Heap top-K ----
import heapq
heap = []
for x in a:
    heapq.heappush(heap, x)
    if len(heap) > k:
        heapq.heappop(heap)

# ---- Dynamic programming (bottom-up, 1D) ----
dp = [base_case] * (n + 1)
for i in range(1, n + 1):
    dp[i] = combine(dp[i - 1], ...)

# ---- Union-Find (disjoint set) ----
parent = list(range(n))
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # path compression
        x = parent[x]
    return x
def union(a, b):
    ra, rb = find(a), find(b)
    if ra != rb:
        parent[ra] = rb

```

9. Complexity Cheat Sheet

Pattern	Typical time	Typical space
Frequency map	$O(n)$	$O(k)$ distinct values
Two Sum (hash map)	$O(n)$	$O(n)$
Two pointers	$O(n)$ (+ $O(n \log n)$ if sorting first)	$O(1)$
Sliding window	$O(n)$	$O(k)$ window state
Prefix sum build + query	$O(n)$ build, $O(1)$ query	$O(n)$
Binary search	$O(\log n)$	$O(1)$
Sort + scan	$O(n \log n)$	$O(1)$ to $O(n)$
Monotonic stack	$O(n)$	$O(n)$
BFS / DFS (graph)	$O(V + E)$	$O(V)$
Backtracking (subsets/permutations)	$O(2^n)$ / $O(n!)$	$O(n)$ recursion depth
Heap top-K	$O(n \log k)$	$O(k)$
Dynamic programming (1D/2D)	$O(n)$ to $O(n*m)$	$O(n)$ to $O(n*m)$, often reducible
Union-Find	$\sim O(\alpha(n))$ per op (near $O(1)$)	$O(n)$

Sorting algorithm	Best	Average	Worst	Space
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Timsort (Python default)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

CP / INTERVIEW: Memorise the two tables above

Almost every 'what's the complexity of your solution?' follow-up question is answered by combining a row from the pattern table with a row from the sorting table.

10. 50 DSA Practice Problems - Solved and Tested

Every solution below was run against multiple test cases (including edge cases) before being printed here - you can copy and run any of them directly. They progress from Beginner (patterns from Section 5 used singly) to Advanced (patterns combined, or applied to trees, graphs and DP).

TIP: How to practice with this list

Cover the code, read only the title. Try to name the pattern and write the solution yourself first. Only then compare against the approach note and code below. Re-attempt any problem you got wrong after two days.

#	Title	Level	Pattern
1	Two Sum	Beginner	Hash map
2	Contains Duplicate	Beginner	Hash set
3	Valid Anagram	Beginner	Frequency map
4	Valid Palindrome	Beginner	Two pointers / filtering
5	First Unique Character	Beginner	Frequency map
6	Missing Number	Beginner	Math / Gauss sum
7	Move Zeroes	Beginner	Two pointers (in place)
8	Valid Parentheses	Beginner	Stack
9	Best Time to Buy/Sell Stock	Beginner	One pass / greedy
10	Merge Two Sorted Lists (arrays)	Beginner	Two pointers
11	Binary Search	Beginner	Binary search
12	Majority Element	Beginner	Boyer-Moore voting
13	Range Sum Query (Immutable)	Beginner	Prefix sum
14	Reverse Words in a String	Beginner	String / split-join
15	Intersection of Two Arrays	Beginner	Hash set
16	Group Anagrams	Intermediate	Hash map + sorting as key
17	Top K Frequent Elements	Intermediate	Heap / frequency map
18	Longest Substring Without Repeating Characters	Intermediate	Sliding window
19	Maximum Subarray (Kadane's Algorithm)	Intermediate	Dynamic programming (1D)
20	Subarray Sum Equals K	Intermediate	Prefix sum + hash map
21	Container With Most Water	Intermediate	Two pointers
22	3Sum	Intermediate	Sort + two pointers
23	Merge Intervals	Intermediate	Sort + scan
24	Daily Temperatures	Intermediate	Monotonic stack
25	Number of Islands	Intermediate	DFS / grid traversal
26	Shortest Path in Binary Matrix	Intermediate	BFS (grid)
27	Search in Rotated Sorted Array	Intermediate	Modified binary search
28	Product of Array Except Self	Intermediate	Prefix / suffix products
29	Subsets	Intermediate	Backtracking
30	Permutations	Intermediate	Backtracking
31	Kth Largest Element in an Array	Intermediate	Min-heap of size k
32	Min Stack	Intermediate	Stack (auxiliary structure)
33	Jump Game	Intermediate	Greedy
34	Meeting Rooms II (min rooms needed)	Intermediate	Sort + heap
35	Reverse Linked List	Intermediate	Linked list / pointer manipulation

#	Title	Level	Pattern
36	Merge k Sorted Lists (arrays)	Advanced	Heap
37	Sliding Window Maximum	Advanced	Monotonic deque
38	Minimum Window Substring	Advanced	Sliding window + frequency map
39	Longest Consecutive Sequence	Advanced	Hash set
40	Trapping Rain Water	Advanced	Two pointers
41	Largest Rectangle in Histogram	Advanced	Monotonic stack
42	Course Schedule (cycle detection)	Advanced	Topological sort (Kahn's BFS)
43	Coin Change (minimum coins)	Advanced	Dynamic programming (1D)
44	Longest Increasing Subsequence	Advanced	Binary search + DP (patience sorting)
45	N-Queens (count solutions)	Advanced	Backtracking with pruning
46	Dijkstra's Shortest Path	Advanced	Heap-based greedy
47	Linked List Cycle Detection	Advanced	Floyd's slow/fast pointers
48	Lowest Common Ancestor (Binary Tree)	Advanced	Recursion (post-order)
49	Edit Distance	Advanced	Dynamic programming (2D)
50	Find Median from Data Stream	Advanced	Two heaps

10.1 Beginner Problems

Problem 1. Two Sum - Pattern: Hash map

Approach: Store each number's index in a dict as you scan. Before storing the current number, check whether its complement ($\text{target} - x$) is already in the dict - if so, you have found the pair.

Complexity: $O(n)$ time, $O(n)$ space

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

Problem 2. Contains Duplicate - Pattern: Hash set

Approach: Duplicates vanish when you build a set. If the set is smaller than the original list, something repeated.

Complexity: $O(n)$ time, $O(n)$ space

```
def contains_duplicate(nums):
    return len(set(nums)) != len(nums)
```

Problem 3. Valid Anagram - Pattern: Frequency map

Approach: Two strings are anagrams exactly when they have identical letter counts. Counter gives an $O(n)$ equality check.

Complexity: $O(n)$ time, $O(1)$ space (26 letters)

```
def is_anagram(s, t):
    return Counter(s) == Counter(t)
```

Problem 4. Valid Palindrome - Pattern: Two pointers / filtering

Approach: Keep only alphanumeric characters, lowercase them, and compare the cleaned string to its reverse.

Complexity: $O(n)$ time, $O(n)$ space

```
def is_palindrome(s):
    t = [c.lower() for c in s if c.isalnum()]
    return t == t[::-1]
```

Problem 5. First Unique Character - Pattern: Frequency map

Approach: Count every character first. Then scan again in order and return the first character whose count is 1.

Complexity: $O(n)$ time, $O(1)$ space

```
def first_unique(s):
    freq = Counter(s)
    for i, c in enumerate(s):
        if freq[c] == 1:
            return i
    return -1
```

Problem 6. Missing Number - Pattern: Math / Gauss sum

Approach: The numbers $0..n$ should sum to $n(n+1)/2$. Subtract the actual sum from that expected sum to find the gap.

Complexity: $O(n)$ time, $O(1)$ space

```
def missing_number(nums):
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)
```

Problem 7. Move Zeroes - Pattern: Two pointers (in place)

Approach: A write pointer tracks where the next non-zero value goes; swap it into place as you scan. Zeros are pushed to the end automatically.

Complexity: $O(n)$ time, $O(1)$ space

```
def move_zeroes(nums):
    w = 0
    for r in range(len(nums)):
        if nums[r] != 0:
            nums[w], nums[r] = nums[r], nums[w]
            w += 1
    return nums
```

Problem 8. Valid Parentheses - Pattern: Stack

Approach: Push opening brackets. On a closing bracket, it must match whatever is on top of the stack, or the string is invalid.

Complexity: $O(n)$ time, $O(n)$ space

```
def valid_parentheses(s):
    pair = {'(': ')', '[': ']', '{': '}'
    stack = []
    for c in s:
        if c in pair:
            if not stack or stack.pop() != pair[c]:
                return False
        else:
            stack.append(c)
    return not stack
```

Problem 9. Best Time to Buy/Sell Stock - Pattern: One pass / greedy

Approach: Track the lowest price seen so far; at each day compute the profit if you sold today, and keep the best.

Complexity: $O(n)$ time, $O(1)$ space

```
def max_profit(prices):
    low = float('inf')
    best = 0
    for p in prices:
        low = min(low, p)
        best = max(best, p - low)
    return best
```

Problem 10. Merge Two Sorted Lists (arrays) - Pattern: Two pointers

Approach: Walk both arrays with a pointer each, always taking the smaller front element, then append whatever remains.

Complexity: $O(n+m)$ time, $O(n+m)$ space

```
def merge_sorted(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i])
            i += 1
        else:
            out.append(b[j])
            j += 1
    out.extend(a[i:])
    out.extend(b[j:])
    return out
```

Problem 11. Binary Search - Pattern: Binary search

Approach: Repeatedly halve the search range by comparing the middle element to the target.

Complexity: $O(\log n)$ time, $O(1)$ space

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Problem 12. Majority Element - Pattern: Boyer-Moore voting

Approach: Keep a candidate and a counter. A matching element increments the counter; a mismatch decrements it, and a new candidate is picked when the counter hits zero. The true majority element always survives.

Complexity: $O(n)$ time, $O(1)$ space

```
def majority_element(nums):
    cand, count = None, 0
    for x in nums:
        if count == 0:
            cand = x
        count += 1 if x == cand else -1
    return cand
```

Problem 13. Range Sum Query (Immutable) - Pattern: Prefix sum

Approach: Precompute prefix sums once. Any range sum is then a subtraction of two prefix values.

Complexity: $O(n)$ build, $O(1)$ per query

```
class RangeSum:
    def __init__(self, nums):
        self.prefix = [0]
        for x in nums:
            self.prefix.append(self.prefix[-1] + x)

    def query(self, l, r):
        # inclusive l..r
        return self.prefix[r + 1] - self.prefix[l]
```

Problem 14. Reverse Words in a String - Pattern: String / split-join

Approach: `split()` with no arguments already collapses multiple spaces and strips ends, so reversing the resulting list and joining solves it directly.

Complexity: $O(n)$ time, $O(n)$ space

```
def reverse_words(s):
    return " ".join(reversed(s.split()))
```

Problem 15. Intersection of Two Arrays - Pattern: Hash set

Approach: Convert both arrays to sets; the `&` operator gives the intersection directly.

Complexity: $O(n+m)$ time, $O(n+m)$ space

```
def intersection(a, b):
    return sorted(set(a) & set(b))
```

10.2 Intermediate Problems

Problem 16. Group Anagrams - Pattern: Hash map + sorting as key

Approach: Words that are anagrams produce the same string when their letters are sorted - use that sorted string as a dict key to group them.

Complexity: $O(n * k \log k)$ time (k = word length), $O(n*k)$ space

```
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[tuple(sorted(w))].append(w)
    return list(groups.values())
```

Problem 17. Top K Frequent Elements - Pattern: Heap / frequency map

Approach: Count frequencies with Counter, then use `heapq.nlargest(k, ...)` to avoid a full $O(n \log n)$ sort when k is small.

Complexity: $O(n \log k)$ time, $O(n)$ space

```
def top_k_frequent(nums, k):
    freq = Counter(nums)
    return heapq.nlargest(k, freq, key=freq.get)
```

Problem 18. Longest Substring Without Repeating Characters - Pattern: Sliding window

Approach: Expand the window with `right`; whenever a repeated character's LAST seen index falls inside the window, jump `left` past it.

Complexity: $O(n)$ time, $O(\min(n, \text{alphabet}))$ space

```
def longest_unique_substring(s):
    last = {}
    left = best = 0
    for right, c in enumerate(s):
        if c in last and last[c] >= left:
            left = last[c] + 1
        last[c] = right
        best = max(best, right - left + 1)
    return best
```

Problem 19. Maximum Subarray (Kadane's Algorithm) - Pattern: Dynamic programming (1D)

Approach: At each position decide: extend the previous subarray, or start fresh here - whichever gives a bigger sum. Track the best seen.

Complexity: $O(n)$ time, $O(1)$ space

```
def max_subarray(nums):
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

Problem 20. Subarray Sum Equals K - Pattern: Prefix sum + hash map

Approach: A subarray sums to k exactly when two prefix sums differ by k . Count how many earlier prefixes equal (current prefix - k).

Complexity: $O(n)$ time, $O(n)$ space

```
def subarray_sum_k(nums, k):
    count = 0
    prefix = 0
    seen = {0: 1}
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count
```

Problem 21. Container With Most Water - Pattern: Two pointers

Approach: Start at both ends. The shorter wall is always the bottleneck, so move that pointer inward - moving the taller one can never help.

Complexity: $O(n)$ time, $O(1)$ space

```
def max_area(h):
    l, r = 0, len(h) - 1
    best = 0
    while l < r:
        best = max(best, min(h[l], h[r]) * (r - l))
        if h[l] < h[r]:
            l += 1
        else:
            r -= 1
    return best
```

Problem 22. 3Sum - Pattern: Sort + two pointers

Approach: Sort the array. Fix one number, then use two pointers on the rest to find pairs that complete the triplet to zero, skipping duplicates.

Complexity: $O(n^2)$ time, $O(1)$ extra space (excl. output)

```
def three_sum(nums):
    nums.sort()
    res = []
    n = len(nums)
    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        l, r = i + 1, n - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0:
                l += 1
            elif s > 0:
                r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                l += 1
                r -= 1
                while l < r and nums[l] == nums[l - 1]:
                    l += 1
    return res
```

Problem 23. Merge Intervals - Pattern: Sort + scan

Approach: Sort by start time. Walk through; if the next interval starts before (or at) the current merged interval's end, extend it; otherwise start a new group.

Complexity: $O(n \log n)$ time, $O(n)$ space

```
def merge_intervals(iv):
    if not iv:
        return []
    iv.sort()
    out = [iv[0]]
    for s, e in iv[1:]:
        if s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

Problem 24. Daily Temperatures - Pattern: Monotonic stack

Approach: Keep indices of days with not-yet-answered (decreasing) temperatures on a stack. A warmer day pops and answers every colder day still waiting.

Complexity: $O(n)$ time, $O(n)$ space

```
def daily_temperatures(t):
    ans = [0] * len(t)
    stack = [] # indices with decreasing temps
    for i, x in enumerate(t):
        while stack and t[stack[-1]] < x:
            j = stack.pop()
            ans[j] = i - j
        stack.append(i)
    return ans
```

Problem 25. Number of Islands - Pattern: DFS / grid traversal

Approach: Scan every cell; whenever an unvisited land cell is found, flood-fill (DFS) the whole connected island and count it once.

Complexity: $O(R \times C)$ time, $O(R \times C)$ space

```
def num_islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen = set()

    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols:
            return
        if grid[r][c] != "1" or (r, c) in seen:
            return
        seen.add((r, c))
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1" and (r, c) not in seen:
                dfs(r, c)
                count += 1
    return count
```

Problem 26. Shortest Path in Binary Matrix - Pattern: BFS (grid)

Approach: BFS from the top-left in 4 directions, tracking distance. The first time BFS reaches the bottom-right cell, that distance is shortest.

Complexity: $O(R \times C)$ time, $O(R \times C)$ space

```
def shortest_path(grid):
    n, m = len(grid), len(grid[0])
    if grid[0][0] == 1 or grid[n - 1][m - 1] == 1:
        return -1
    q = deque([(0, 0, 1)])
    seen = {(0, 0)}
    while q:
        r, c, d = q.popleft()
        if (r, c) == (n - 1, m - 1):
            return d
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] == 0 and (nr, nc) not in seen:
                seen.add((nr, nc))
                q.append((nr, nc, d + 1))
    return -1
```

Problem 27. Search in Rotated Sorted Array - Pattern: Modified binary search

Approach: At each mid-point, one half of the array is always properly sorted. Check whether the target falls in that sorted half; otherwise search the other half.

Complexity: $O(\log n)$ time, $O(1)$ space

```
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:          # left half sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                             # right half sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Problem 28. Product of Array Except Self - Pattern: Prefix / suffix products

Approach: First pass fills each position with the product of everything to its LEFT; second pass (right to left) multiplies in the product of everything to its RIGHT.

Complexity: $O(n)$ time, $O(1)$ extra space (excl. output)

```
def product_except_self(nums):
    n = len(nums)
    res = [1] * n
    left = 1
    for i in range(n):
        res[i] = left
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):
        res[i] *= right
        right *= nums[i]
    return res
```

Problem 29. Subsets - Pattern: Backtracking

Approach: At each element make a binary choice - include it or not - and recurse. Every leaf of this decision tree is one subset.

Complexity: $O(2^n * n)$ time, $O(n)$ recursion depth

```
def subsets(nums):
    res = []
    path = []

    def bt(i):
        if i == len(nums):
            res.append(path[:])
            return
        bt(i + 1)          # skip nums[i]
        path.append(nums[i]) # take nums[i]
        bt(i + 1)
        path.pop()

    bt(0)
    return res
```

Problem 30. Permutations - Pattern: Backtracking

Approach: Build the path one element at a time, marking used elements so each appears exactly once per permutation, then undo before trying the next choice.

Complexity: $O(n! * n)$ time, $O(n)$ recursion depth

```
def permutations(nums):
    res = []
    path = []
    used = [False] * len(nums)

    def bt():
        if len(path) == len(nums):
            res.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            bt()
            path.pop()
            used[i] = False

    bt()
    return res
```

Problem 31. Kth Largest Element in an Array - Pattern: Min-heap of size k

Approach: Push every element into a min-heap; whenever the heap exceeds size k, pop the smallest. What remains on top is the k-th largest.

Complexity: $O(n \log k)$ time, $O(k)$ space

```
def kth_largest(nums, k):
    heap = []  # min-heap holding the k largest
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
```

Problem 32. Min Stack - Pattern: Stack (auxiliary structure)

Approach: Keep a second stack that always tracks the minimum-so-far alongside the main stack, so get_min never has to scan.

Complexity: $O(1)$ time per operation, $O(n)$ space

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.mins = []

    def push(self, x):
        self.stack.append(x)
        self.mins.append(x if not self.mins else min(x, self.mins[-1]))

    def pop(self):
        self.mins.pop()
        return self.stack.pop()

    def get_min(self):
        return self.mins[-1]
```

Problem 33. Jump Game - Pattern: Greedy

Approach: Track the farthest index reachable so far. If you ever reach a position beyond that frontier before updating it, the end is unreachable.

Complexity: $O(n)$ time, $O(1)$ space

```
def can_jump(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True
```

Problem 34. Meeting Rooms II (min rooms needed) - Pattern: Sort + heap

Approach: Sort meetings by start time. Keep a min-heap of end times for rooms in use; if the earliest-ending room is free before the next meeting starts, reuse it - otherwise open a new room.

Complexity: $O(n \log n)$ time, $O(n)$ space

```
def min_meeting_rooms(iv):
    iv.sort()
    ends = [] # min-heap of end times
    for s, e in iv:
        if ends and ends[0] <= s:
            heapq.heapreplace(ends, e)
        else:
            heapq.heappush(ends, e)
    return len(ends)
```

Problem 35. Reverse Linked List - Pattern: Linked list / pointer manipulation

Approach: Walk the list once, at each node saving next before rewiring `cur.next` to point backward to `prev`.

Complexity: $O(n)$ time, $O(1)$ space

```
def reverse_list(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

10.3 Advanced Problems

Problem 36. Merge k Sorted Lists (arrays) - Pattern: Heap

Approach: Keep one 'current smallest candidate' per list in a min-heap. Pop the overall smallest, output it, then push that list's next element.

Complexity: $O(N \log k)$ time (N = total elements), $O(k)$ space

```
def merge_k_sorted(lists):
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heap.append((lst[0], i, 0))
    heapq.heapify(heap)
    out = []
    while heap:
        val, i, j = heapq.heappop(heap)
        out.append(val)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out
```

Problem 37. Sliding Window Maximum - Pattern: Monotonic deque

Approach: Keep indices in the deque with DECREASING values. Drop indices that fall outside the window and any smaller values at the back before adding a new one - the front is always the window's max.

Complexity: $O(n)$ time, $O(k)$ space

```
def sliding_window_max(nums, k):
    dq = deque()          # indices, values decreasing
    out = []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out
```

Problem 38. Minimum Window Substring - Pattern: Sliding window + frequency map

Approach: Expand right until the window contains every character of t with the right multiplicity, then shrink left as far as possible while still valid, recording the smallest such window.

Complexity: $O(n + m)$ time, $O(m)$ space

```
def min_window(s, t):
    need = Counter(t)
    missing = len(t)
    best = (float('inf'), 0, 0)
    left = 0
    for right, c in enumerate(s):
        if need[c] > 0:
            missing -= 1
            need[c] -= 1
        if missing == 0:
            while need[s[left]] < 0:          # drop useless chars
                need[s[left]] += 1
                left += 1
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            need[s[left]] += 1                # break the window
            missing += 1
            left += 1
    return "" if best[0] == float('inf') else s[best[1]:best[2] + 1]
```

Problem 39. Longest Consecutive Sequence - Pattern: Hash set

Approach: Put all numbers in a set. Only start counting a run from a number whose predecessor (x-1) is absent - that guarantees each run is counted exactly once, giving $O(n)$ overall despite the inner while loop.

Complexity: $O(n)$ time, $O(n)$ space

```
def longest_consecutive(nums):
    s = set(nums)
    best = 0
    for x in s:
        if x - 1 not in s:          # x starts a run
            y = x
            while y + 1 in s:
                y += 1
            best = max(best, y - x + 1)
    return best
```

Problem 40. Trapping Rain Water - Pattern: Two pointers

Approach: Water above any bar is limited by the shorter of the tallest wall to its left and right. Move the pointer on the shorter side inward, since that side's max is the true limiting wall.

Complexity: $O(n)$ time, $O(1)$ space

```
def trap(h):
    l, r = 0, len(h) - 1
    lmax = rmax = 0
    water = 0
    while l < r:
        if h[l] < h[r]:
            lmax = max(lmax, h[l])
            water += lmax - h[l]
            l += 1
        else:
            rmax = max(rmax, h[r])
            water += rmax - h[r]
            r -= 1
    return water
```

Problem 41. Largest Rectangle in Histogram - Pattern: Monotonic stack

Approach: Keep bar indices on a stack with increasing heights. When a shorter bar appears, pop taller bars and compute the rectangle each could have formed, extending back to the last still-shorter bar.

Complexity: $O(n)$ time, $O(n)$ space

```
def largest_rectangle(h):
    stack = []  # (start_index, height), heights increasing
    best = 0
    h = h + [0]  # sentinel flushes the stack
    for i, x in enumerate(h):
        start = i
        while stack and stack[-1][1] >= x:
            idx, height = stack.pop()
            best = max(best, height * (i - idx))
            start = idx
        stack.append((start, x))
    return best
```

Problem 42. Course Schedule (cycle detection) - Pattern: Topological sort (Kahn's BFS)

Approach: Build a graph of prerequisites and count in-degrees. Repeatedly remove nodes with in-degree 0; if every course gets removed, there is no cycle and the schedule is possible.

Complexity: $O(V+E)$ time, $O(V+E)$ space

```
def can_finish(n, prereqs):
    graph = defaultdict(list)
    indeg = [0] * n
    for a, b in prereqs:  # b must be taken before a
        graph[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    done = 0
    while q:
        u = q.popleft()
        done += 1
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return done == n
```

Problem 43. Coin Change (minimum coins) - Pattern: Dynamic programming (1D)

Approach: $dp[a]$ = fewest coins to make amount a . For each amount, try every coin and take the best $dp[a - \text{coin}] + 1$. This is where greedy (Section 5.13) can fail but DP always finds the optimum.

Complexity: $O(\text{amount} * \text{coins})$ time, $O(\text{amount})$ space

```
def coin_change(coins, amount):
    INF = float('inf')
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
    return -1 if dp[amount] == INF else dp[amount]
```

Problem 44. Longest Increasing Subsequence - Pattern: Binary search + DP (patience sorting)

Approach: Maintain the smallest possible 'tail' value for an increasing subsequence of each length. Binary-search where each new number fits; the final length of this tails array is the LIS length.

Complexity: $O(n \log n)$ time, $O(n)$ space

```
def lis(nums):
    tails = []
    for x in nums:
        i = bisect.bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
    return len(tails)
```

Problem 45. N-Queens (count solutions) - Pattern: Backtracking with pruning

Approach: Place queens row by row. Track occupied columns and the two diagonal directions in sets so each placement check is $O(1)$; backtrack when no column is safe.

Complexity: $O(n!)$ time (heavily pruned in practice), $O(n)$ space

```
def n_queens(n):
    cols, d1, d2 = set(), set(), set()

    def bt(r):
        if r == n:
            return 1
        total = 0
        for c in range(n):
            if c in cols or (r - c) in d1 or (r + c) in d2:
                continue
            cols.add(c)
            d1.add(r - c)
            d2.add(r + c)
            total += bt(r + 1)
            cols.remove(c)
            d1.remove(r - c)
            d2.remove(r + c)
        return total

    return bt(0)
```

Problem 46. Dijkstra's Shortest Path - Pattern: Heap-based greedy

Approach: Always expand the closest not-yet-finalised node next (via a min-heap keyed by distance). Relax its edges; skip a popped entry if a shorter distance was already found for that node (stale entry).

Complexity: $O((V+E) \log V)$ time, $O(V)$ space

```
def dijkstra(graph, src):
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, float('inf')): # stale entry
            continue
        for v, w in graph.get(u, []):
            nd = d + w
            if nd < dist.get(v, float('inf')):
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    return dist
```

Problem 47. Linked List Cycle Detection - Pattern: Floyd's slow/fast pointers

Approach: Two pointers move through the list at different speeds. If there is a cycle, the fast pointer eventually laps the slow one and they meet; if the list ends, there is no cycle.

Complexity: $O(n)$ time, $O(1)$ space

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False
```

Problem 48. Lowest Common Ancestor (Binary Tree) - Pattern: Recursion (post-order)

Approach: Recurse into both subtrees. If a node is one of the two targets, or both targets are found in different subtrees, that node is the LCA.

Complexity: $O(n)$ time, $O(h)$ space (h = tree height)

```
def lca(root, p, q):
    if root is None or root is p or root is q:
        return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    if left and right:
        return root
    return left or right
```

Problem 49. Edit Distance - Pattern: Dynamic programming (2D)

Approach: $dp[i][j]$ = edits to turn the first i characters of a into the first j characters of b . Matching characters cost nothing; otherwise take the best of insert, delete, or replace.

Complexity: $O(n*m)$ time, $O(n*m)$ space

```
def edit_distance(a, b):
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[n][m]
```

Problem 50. Find Median from Data Stream - Pattern: Two heaps

Approach: Keep a max-heap for the smaller half of numbers and a min-heap for the larger half, rebalanced after every insert so their sizes never differ by more than one. The median sits at the top of one or both heaps.

Complexity: $O(\log n)$ time per insert, $O(1)$ per query, $O(n)$ space

```
class MedianFinder:
    def __init__(self):
        self.lo = []          # max-heap (store negatives)
        self.hi = []          # min-heap

    def add(self, x):
        heapq.heappush(self.lo, -x)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))

    def median(self):
        if len(self.lo) > len(self.hi):
            return -self.lo[0]
        return (-self.lo[0] + self.hi[0]) / 2
```

11. Closing Notes

You now have a working vocabulary that spans imports and file I/O, the standard-library modules used constantly in real code and contests, hand-built implementations of every core data structure, fourteen named problem-solving patterns, formal complexity analysis, and fifty tested solutions spanning three difficulty levels.

Suggested revision order

- Week 1: Sections 1-4 (modules, files, standard library, data structure implementations).
- Week 2: Section 5 patterns - implement each template from memory, then solve 2-3 Beginner problems per pattern.
- Week 3: Section 6 complexity - be able to state time/space complexity for anything you write, unprompted.
- Week 4 onward: Section 10 Intermediate then Advanced problems, timed at 20-25 minutes each before checking the solution.

KEY IDEA: Coming in Part 7

OOP-heavy interview design questions (e.g. designing a parking lot, LRU cache, rate limiter), and an introduction to concurrency (threading, multiprocessing, asyncio) for the software-engineering side of the notes series.

Python Master Notes - Part 6 of the series. Parts 1-5 cover fundamentals through OOP and exceptions.